

Compiler Design (IT38516)

8 February 2026

Manjeet Kumar Soni
Associate Professor, IT

IT38516- Compiler Design

Scheme of Examination

HOURS PER WEEK			CREDITS			MAXIMUM MARKS				
T	P	Tu	T	P	Tu	THEORY		PRACTICAL		TOTAL MARKS
						CW	END SEM	SW	END SEM	
3	-	-	3	-	-	30	70	-	-	100

Class Work Marks (CW)

Two Mid Sem Test - 20 Marks Each

Best of two/ Average of two

Quiz / Assignments - 5 Marks

Regularity - 5 Marks

IT38516- Compiler Design

PRE-REQUISITES: Theory of computation, C Language

COURSE OBJECTIVES:

1. Learn the design principles of a Compiler and different levels of translation.
2. To understand, design and implement a lexical analyzer.
3. To understand, design and implement a parser.
4. To understand the type conversions.
5. Learn how to optimize and effectively generate machine codes.

IT38516- Compiler Design

COURSE OUTCOMES: After completion of course, students will be able to:

1. Explain the phases of a compiler and the role of lexical analysis using tools like LEX.
2. Construct context-free grammars and apply parsing techniques including predictive and LR.
3. Design syntax-directed definitions and analyze type systems and runtime environments.
4. Generate intermediate code and apply flow graph-based code generation and optimizations.
5. Evaluate and optimize generated code using data flow analysis and loop transformations.

IT38516- Compiler Design

TEXT BOOKS RECOMMENDED:

1. **A. V. Aho, R. Sethi, and J. D. Ullman. “Compilers: Principles, Techniques and Tools”, Pearson Education.**
2. **Keith D Cooper and Linda Torczon, “Engineering a Compiler”, Morgan Kaufmann Publishers Elsevier Science.**

REFERENCE BOOKS:

1. **Louden, “Compiler Construction: Principles and Practice”, Cengage Learning.**
2. **A. C. Holub, “Compiler Design in C” , Prentice-Hall Inc., 1993.**
3. **Mak, “writing compiler & Interpreters”, Willey Pub.**

IT38516- Compiler Design

COURSE CONTENTS:

UNIT1: Introduction to compiling & Lexical Analysis: Introduction of Compiler, Major data Structure in compiler, BOOT Strapping & Porting, Compiler structure: analysis-synthesis model of compilation, various phases of a compiler, Lexical analysis: Input buffering , Specification & Recognition of Tokens, LEX.

IT38516- Compiler Design

COURSE CONTENTS:

**UNIT2: Syntax Analysis &Syntax Directed
Translation,Syntax analysis: CFGs, Top down parsing,
Brute force approach, recursive descent parsing,
transformation on the grammars, predictive parsing,
bottom up parsing, operator, precedence parsing, LR
parsers (SLR,LALR, Canonical LR), Parser generation.**

IT38516- Compiler Design

COURSE CONTENTS:

UNIT3: Syntax directed definitions: Construction of Syntax trees, Bottom up evaluation of S-attributed definition, L-attribute definition, Top down translation, Bottom Up evaluation of inherited attributes, Recursive Evaluation, Analysis of Syntax directed definition. Type Checking & Run Time Environment: Type checking: type system, specification of simple type checker, equivalence of expression, types, type conversion, overloading of functions and operations, polymorphic functions. Run time Environment: storage organization, Storage allocation strategies, parameter passing, dynamic storage allocation , Symbol table.

IT38516- Compiler Design

COURSE CONTENTS:

UNIT4: Code Generation: Intermediate code generation: Declarations, Assignment statements, Boolean expressions, Case statements, Back patching, Procedure calls Code Generation: Issues in the design of code generator, Basic block and flow graphs, Register allocation and assignment, DAG representation of basic blocks, peephole optimization, generating code from DAG.

IT38516- Compiler Design

COURSE CONTENTS:

UNIT5: Code Optimization: Introduction to Code optimization: sources of optimization of basic blocks, loops in flow graphs, dead code elimination, loop optimization, Introduction to global data flow analysis, Code Improving transformations ,Data flow analysis of structure flow graph Symbolic debugging of optimized code.

IT38516- Compiler Design

LECTURE PLAN

S. No.	Unit	Total Lectures
1	UNIT 1	7
2	UNIT 2	10
3	UNIT 3	5
4	UNIT 4	5
5	UNIT 5	5
	Total	31

IT38516- Compiler Design

LECTURE PLAN

S. No.	Unit 1	Total Lectures
1	Introduction to Compiler, Data Structures in Compiler	0.5
2	Different Phases of Compiler	0.5
3	Bootstrapping, Cross compiler, single pass and multi pass compiler, Token Patterns and Lexems	1
4	Review of language, Automata, and Regular Expression	0.3
5	Generalized transition diagram, Transition diagrams for programming language constructs	0.3
6	RE to DFA using Syntax tree method	0.4
7	Introduction to LEX- a lexical analyzer generator	1
8	Writing LEX source programs using regular Expressions	1
9	Discussion on problems that can constructed on lexical analysis	1
10	Discussion on problems that can constructed on lexical analysis	1

Introduction to Compiler

Why we Study Compiler?

- A compiler is an excellent example of theory translated to the practice
- Modern Applications for compiler technology
 - **Parsers for HTML in web browser**
 - **Interpreters for javascripts/flash**
 - **Machine code generation for high level languages**
 - **Software testing**
 - **Program Optimization**
 - **Malicious code detection**
 - **Design of new Computer architecture**

Introduction to Compiler

Compiler Uses algorithms and techniques from a very large number of areas in computer science

- Draw results from mathematical logic, lattice theory, linear algebra, probability, etc.

- **Type checking, static analysis, dependence analysis and loop parallelization etc.**

- Makes Practical Application of

- **Greedy algorithms - register allocation**
- **Heuristic Search - list scheduling**
- **Graph algorithms - dead code elimination, register allocation**
- **Dynamic Programming - instruction selection**
- **Optimization techniques - instruction scheduling**
- **Finite automata - lexical Analysis**
- **Pushdown automata - parsing**
- **Fixed point algorithms - data flow analysis**
- **Complex data structures - symbol table, parse tree, data dependence graph**
- **Computer architecture - machine code generation**

Introduction to Compiler

Other Uses of Scanning and Parsing Technique

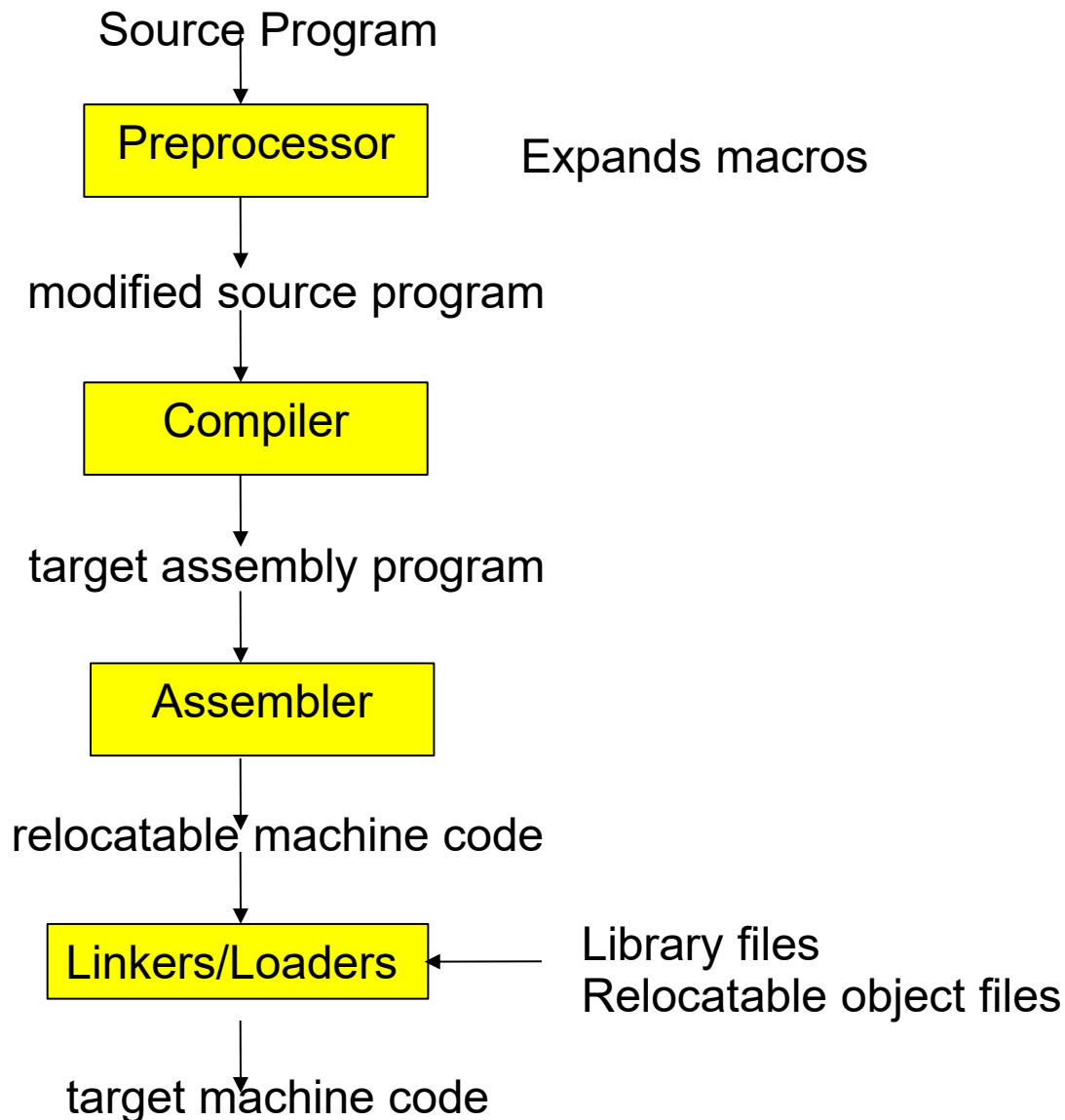
- **Assembler implementation**
- **Online text searching (GREP, AWK) and word processing**
- **Website filtering**
- **Command language interpreter**
- **Scripting language interpretation (Unix shell, Perl, Python)**
- **XML parsing and document tree construction**
- **Database query interpreters**

Introduction to Compiler

Other Uses of Program Analysis Techniques

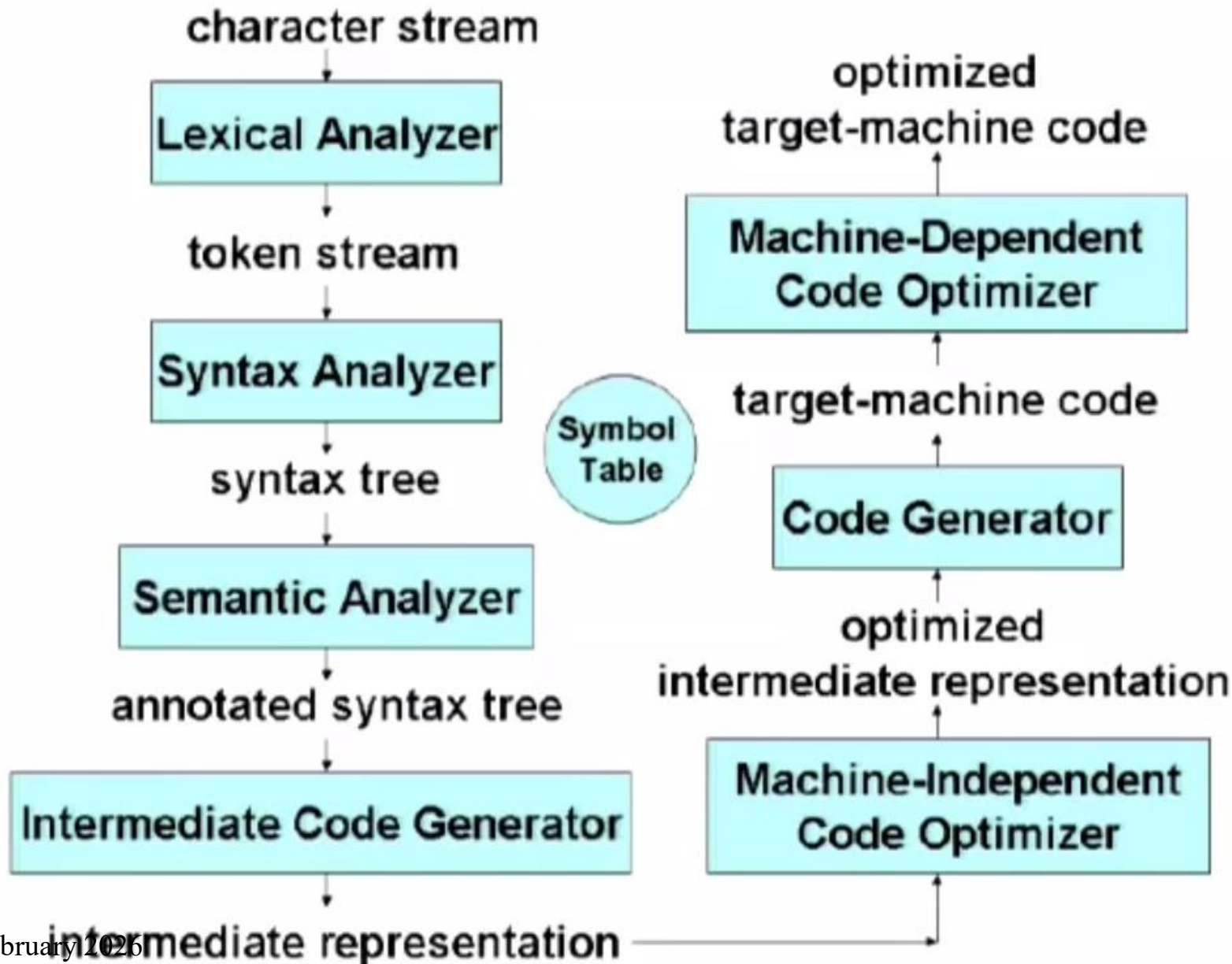
- **Converting a sequential loop to a parallel loop**
- **Program analysis to determine if program are data-race free**
- **Profiling program to determine busy regions**
- **Program slicing**
- **Data-flow analysis approach to software testing**
 - **Uncovering errors along all path**
 - **Dereferencing null pointers**
 - **Buffer overflows and memory leaks**
- **Worst Case Execution Time (WCET) estimation and energy analysis**

Introduction to Compiler



A Language Processing Systems

Introduction to Compiler



Introduction to Compiler

Compilers and Interpreters

- **Compilers generate machine code, whereas interpreters interpret intermediate code**
- **Interpreters are easier to write and provide better error messages (symbol table is still available)**
- **Interpreters are at least 5 times slower than machine code generated by compilers**
- **Interpreters also require much more memory than machine code generated by compilers**
- **Examples: perl, Python, Unix Shell, Java, BASIC, LISP**

Introduction to Compiler

Translation Overview – Lexical Analysis

fahrenheit = centigrade * 1.8 + 32

Lexical Analyzer

<id,1> <assign> <id,2> <multop>
<fconst, 1.8> <addop> <iconst,32>

Syntax Analyzer

Introduction to Compiler

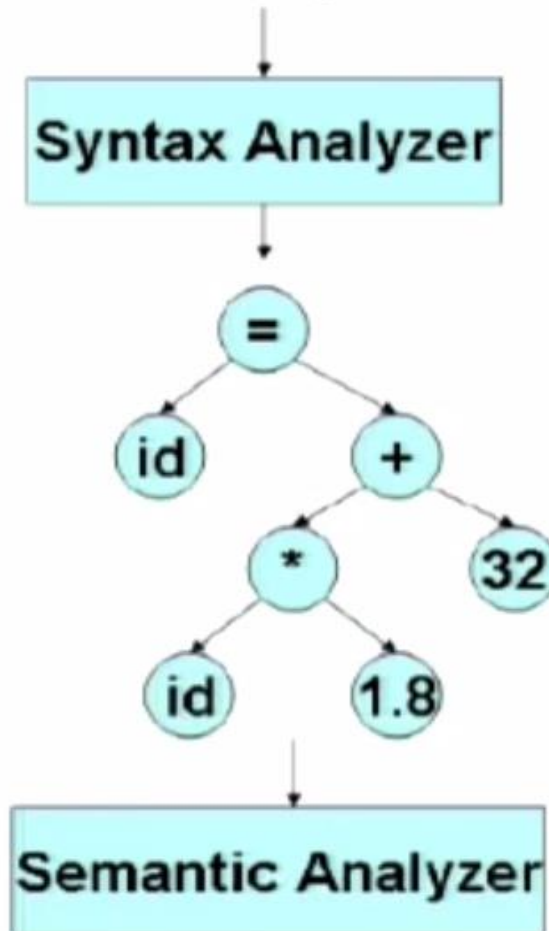
Why Lexical Analysis ?

- **Lexical Analyzer can be generated automatically from regular expression specifications**
 - **LEX and FLEX are two such tools**
- **Lexical Analyzer is a deterministic finite automaton**
- **Why is Lexical Analysis is separate from parsing**
 - **Simplification of Design – software engineering reason**
 - **I/O issues are limited in LA alone**
 - **LA based on finite automata are more efficient to implement than pushdown automata used for parsing(due to stack)**

Introduction to Compiler

Translation Overview – Syntax Analysis

`<id,1> <assign> <id,2> <multop>`
`<fconst, 1.8> <addop> <iconst,32>`



Introduction to Compiler

Parsing or Syntax Analysis

- **Syntax Analyzers (Parser) can be generated automatically from several variants of context-free grammar specifications**
 - LL(1), and LALR(1) are the most popular ones
 - ANTLR(for LL(1), YACC and BISON (for LALR(1)) are such tools
- **Parsers are deterministic push down automata**
- **Parsers cannot handle context-sensitive features of programming languages; e.g.,**
 - Variables are declared before use
 - Types match on both sides of assignment
 - Parameters types and numbers match in declaration and use

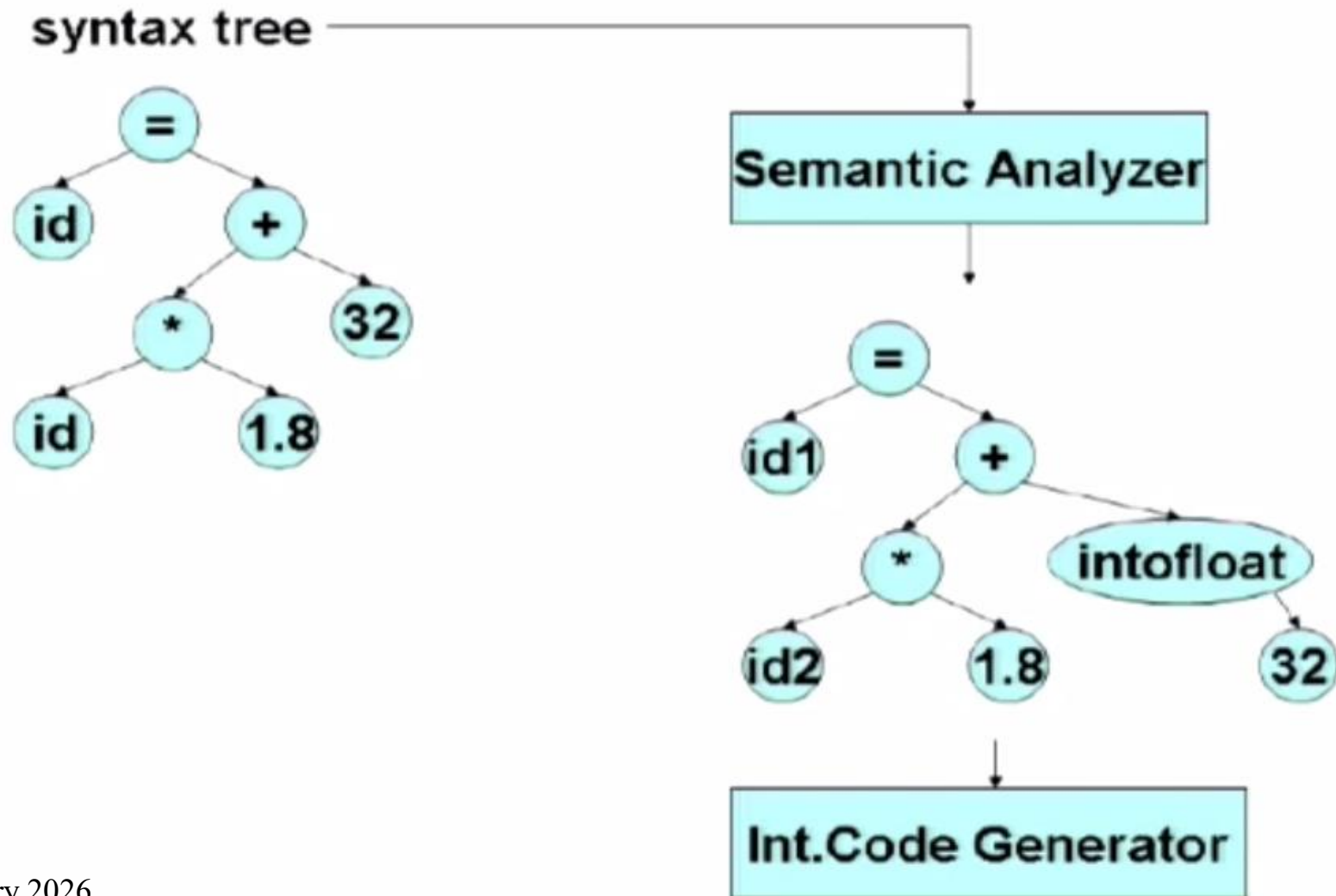
Compiler Design (IT38506)

Lecture – II

Introduction to Compiler

Introduction to Compiler

Translation Overview – Semantic Analysis



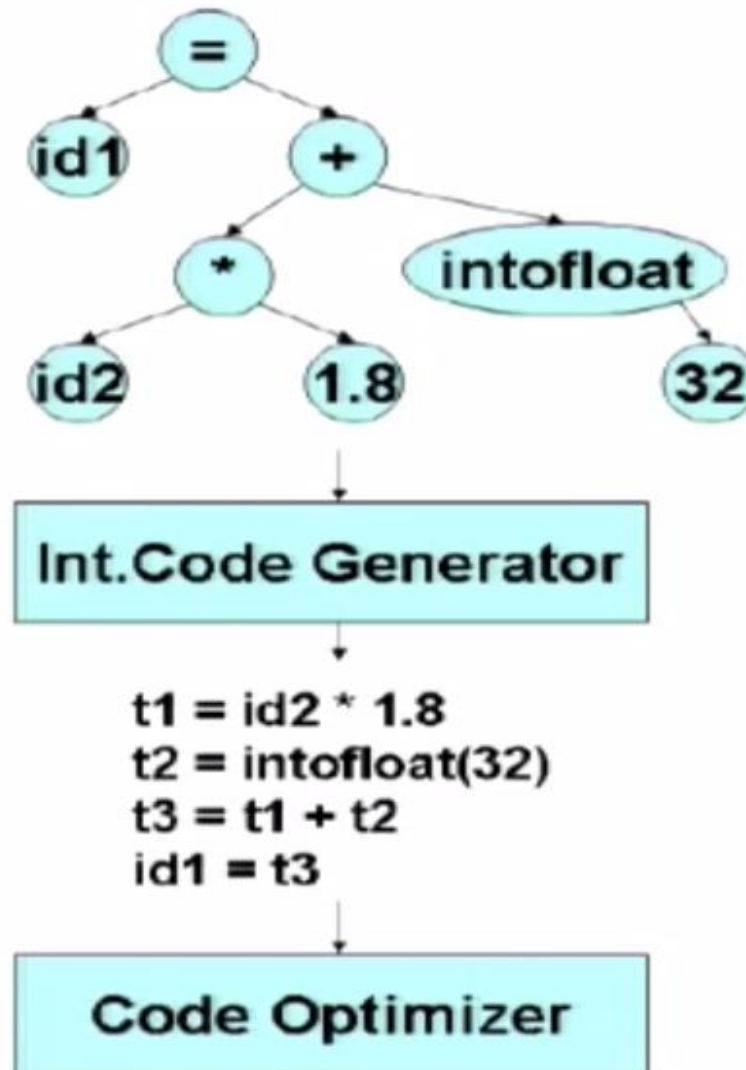
Introduction to Compiler

Semantic Analysis

- **Semantic consistency that cannot be handled at the parsing stage is handled here**
- **Type checking of various programming language constructs is one of the most important tasks**
- **Stores type information in the symbol table or the syntax tree**
 - **Types of variables, function parameters, array dimensions etc.**
 - **Used not only for semantics validation but also for subsequent phases of compilation**
- **Static semantics of programming languages can be specified using attribute grammars**

Introduction to Compiler

Translation Overview – Intermediate Code Generation



Introduction to Compiler

Intermediate Code Generation

- While generating machine code directly from source code is, it entails two problems
 - With m languages and n target machines, we need to write $m \times n$ compilers
 - The code optimizer which is one of the largest and very-difficult-to-write components of any compiler cannot be reused.
- By converting source code to an intermediate code, a machine-independent code optimizer may be written
- Intermediate code must be easy to produce and easy to translate to machine code
 - A sort of universal assembly language
 - Should not contain any machine-specific parameters (register, addresses, etc.)

Introduction to Compiler

Different Types of Intermediate Code

- The type of intermediate code deployed is based on the application
- Quadruples, triples, Indirect triples, abstract syntax trees are the classical forms used for machine-independent optimizations and code generation
- Static Single Assignment for (SSA) is a recent form and enables more effective optimizations (GCC compiler)
 - Conditional constant propagation and global value numbering are more effective on SSA
- Program Dependence Graph (PDG) is useful in automatic parallelization, instruction scheduling and software pipe lining

Introduction to Compiler

Translation Overview – Code Optimization

```
t1 = id2 * 1.8  
t2 = intofloat(32)  
t3 = t1 + t2  
id1 = t3
```

Code Optimizer

```
t1 = id2 * 1.8  
id1 = t1 + 32.0
```

Code Generator

Introduction to Compiler

Machine Independent Code Optimization

- **Intermediate code generation process introduces many inefficiencies**
 - **Extra copies of variables, using variables instead of constants, repeated evaluation of expressions, etc**
- **Code optimization removes such inefficiencies and improves code**
- **Improvement may be time, space, or power consumption**
- **It changes the structure of programs, sometimes of beyond recognition**
 - **Inline functions, unrolls loops, eliminates some programmer-defined variables, etc.**
- **Code optimization consists of a bunch of heuristics and percentage of improvement depends on program(may be zero also)**

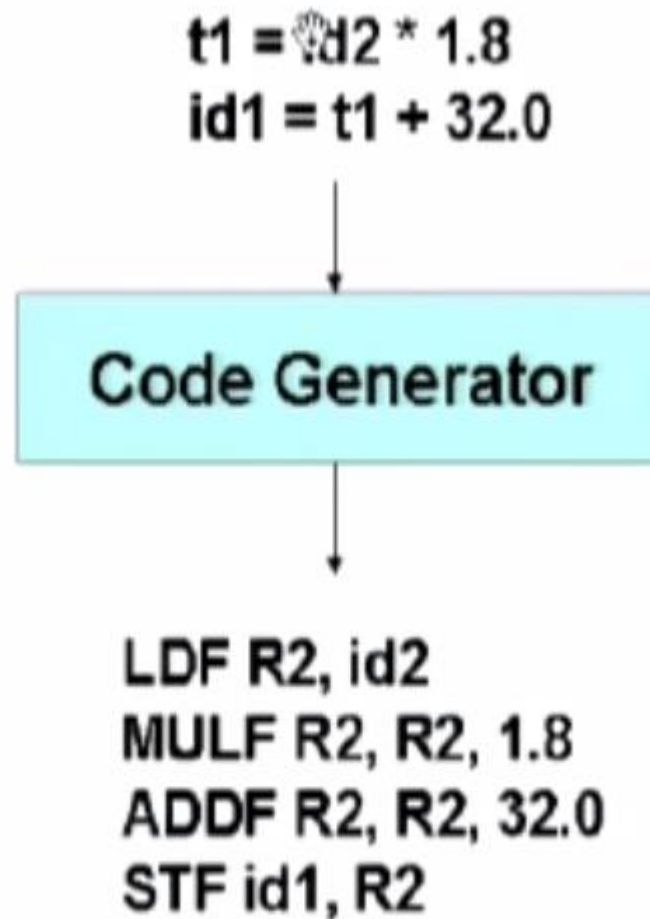
Introduction to Compiler

Examples of Machine-Independent Optimizations

- **Common sub-expression elimination**
- **Copy propagation**
- **Loop invariant code motion**
- **Partial redundancy elimination**
- **Induction variable elimination and strength reduction**
- **Code optimization needs information about the program**
 - which expressions are being recomputed in a function?
 - which definition reach a point?
- **All such information is gathered through data-flow analysis**

Introduction to Compiler

Translation Overview – Code Generation



Introduction to Compiler

Code Generation

- **Converts intermediate code to machine code**
- **Each intermediate code instruction may result in many machine instructions or vice-versa**
- **Must handle all aspects of machine architecture**
 - **Registers, pipelining, cache, multiple function units, etc.**
- **Generating efficient code is an NP-complete problem**
 - **Tree pattern matching-based strategies are among the best**
 - **Needs tree intermediate code**
- **Storage allocation decisions are made here**
 - **Register allocation and assignment are the most important problems**

Introduction to Compiler

Machine dependent Optimizations

- **Peephole optimizations**
 - Analyze sequence of instructions in a small window (peephole) and using preset patterns, replace them with a more efficient sequence
 - Redundant instruction elimination e.g. replace the sequence
$$[LD\ A, R1][ST\ R1, A] \quad \text{by} \quad [LD\ A, R1]$$
 - Eliminate “jump to jump” instructions
 - Use machine idioms (use INC instead of LD and ADD)
- Instruction scheduling (reordering) to eliminate pipeline interlocks and to increase parallelism
- Trace scheduling to increase the size of basic blocks and increase parallelism
- Software pipelining to increase parallelism in loops

Introduction to Compiler

- **Token: A token is a sub string of the source program that is to be treated as a single unit**
 - **The blank separating the characters of these tokens would be normally be eliminated during lexical analysis**
 - ex: identifiers, keywords, constants, operators, punctuation symbol, labels**
- **Pair (Token type, token value)**

Introduction to Compiler

Symbol Table Management

- **Record the identifiers used in the source program**
- **Collect information about various attributes of each identifier**
 - **Storage allocation for identifier**
 - **type**
 - **scope**
 - **procedure name (name and type of its argument, method of passing argument, and type returned if any)**
- **A Symbol table is a data structure containing a record for each identifier, with fields for the attributes of the identifier**

Introduction to Compiler

Error Detection and Reporting

- **Each phase can encounter errors.**
- **After detecting error, a phase must somehow deal with that error, so that compilation can proceed, allowing for the errors in the source program to be detected.**
 - **lexical phase characters do not form any token**
 - **syntax phase token stream violates the structure rules(syntax)**
 - **semantic phase syntactically correct but no meaning to the operation**